MC970 - Introdução à programação paralela
Relatório - Projeto Final
Paralelização do Jogo da Vida de Conway com OpenMP

Daniel de Sousa Cipriano RA: 233228
José Victor Santana Barbosa RA: 245511
Ricardo Emanuel de Oliveira RA: 200518

Junho 2025

1 Descrição de sistema

Os benchmarks foram executados em um sistema Linux baseado em Ubuntu com a seguinte configuração:

Componente	**Descrição**
Sistema operacional	Linux Mint 64 bits
Processador	AMD Ryzen 5 5500U com SMT
Núcleos	6 físicos, 12 lógicos
Clock	4.05 GHz
Memória RAM	16 GB DDR4

2 Ferramentas

OpenMP: O OpenMP(Open Multi-Processing) é uma API para programação de sistemas multi-processados com memória compartilhada. A interface fornece diretivas de compilação, bibliotecas de funções e variáveis de ambiente que facilitam e padronizam a implementação de sistemas com multithreading[1].

3 Introdução

3.1 Autômatos celulares

Autômatos celulares são modelos discretos de computação, estudados na teoria dos autômatos. Um autômato celular define uma grade de células, que pode estar em qualquer número finito de dimensões, com células em estado ligado ou desligado. Para cada célula, é definido um conjunto de

células adjacentes, chamado de vizinhança da célula. Para autômatos bidimensionais, os dois modelos de vizinhança mais comuns são o modelo de vizinhança de Von Neumann, que define quatro células adjacentes como vizinhas, sem incluir as células na diagonal, e o modelo de vizinhança de Moore, que inclui as células na diagonal e define oito células adjacentes como vizinhas. São implementadas regras que definem um padrão para mudança de estado das células, geralmente baseadas em uma função matemática, que varia de acordo com uma passagem discreta de tempo, denominada como avanço de gerações[2]. Os autômatos celulares possuem três características básicas em comum, são elas:

- **Uniformidade:** Todas as céulas são atualizadas pelo mesmo conjunto de regras.
- **Sincronicidade:** Todas as células são atualizadas simultaneamente (existem algumas exceções, como o autômato celular assíncrono).
- **Localidade:** As regras são aplicadas localmente, ou seja, cada célula depende apenas da sua vizinhança.

Na década de 80, o cientista da computação Stephen Wolfram desenvolveu uma classificação para os autômatos celulares. Essa divisão foi baseada na evolução comportamental de cada autômato. Foram definidas quatro classes distintas, sendo que cada autônomo pode se enquadrar em uma ou mais classes simultaneamente, são elas:

- **Classe I:** Autômatos com comportamento constante, que estabilizam em um estado e não possuem evolução continuamente dinâmica.
- **Classe II:** Autômatos com estruturas que mantém um padrão de repetição.
- **Classe III:** Autômatos que possuem comportamento aleatório, com evolução dinâmica e ”caótica”. Esse padrão é utilizado no estudo de aleatoriedade em computação.
- **Classe IV:** Autômatos que evoluem para estruturas complexas e duradouras, mesclando comportamentos estáticos e dinâmicos que podem interagir mutuamente. Esse padrão é definido como computacionalmente universal por Wolfram e pode simular uma máquina de Turing.

3.2 Jogo da vida de Conway

O Jogo da Vida é um autômato celular, desenvolvido pelo matemático britânico John Horton Conway em 1970. O Jogo da Vida de Conway utiliza o modelo de vizinhança de Moore, define os estados das células como viva ou morta e implementa quatro regras para mudança de estado, são elas:

- Se uma célula viva possui menos de duas células vizinhas vivas, então ela estará morta na próxima geração (morte por isolamento).
- Se uma célula viva possui 2 ou 3 células vizinhas vivas, então ela continuará viva na próxima geração.

- Se uma célula viva possui mais de 3 células vizinhas vivas, então ela estará morta na próxima geração (morte por superpopulação).
- Se uma célula morta possui exatamente 3 células vizinhas, então ela estará viva na próxima geração.

O Jogo da Vida de Conway foi o primeiro autômato celular a trazer visibilidade para o tema fora do ambiente acadêmico, trazendo aplicações práticas, através da elaboração de modelos semelhantes, em diversas áreas como biologia, medicina, física, química, artes, ciências sociais e etc. O jogo da vida de Conway é considerado um autômato celular de classe IV e os padrões estruturais gerados pelo autômato são denominados como formas de vida. Os padrões do Jogo da Vida são subdivididos em grupos, são eles:

- **Still lifes:** são padrões estáticos, que param de oscilar após um pequeno número de gerações. Alguns exemplos de still lifes são o Block, o Tub, o Beehive, o Ship, o Snake, o Pond, o Fishhook e o Loaf, que podem ser vistos respectivamente na Figura 1.
- **Oscilators:** são padrões que repetem constantemente uma sequência de estruturas, retornando ao estado original sempre no fim do loop. Um exemplo de oscilator é o Pulsar, que pode ser visto dividido em fases na Figura 2
- **Spaceships:** são padrões que, assim como os oscilators, seguem uma variação periódica, com o acréscimo de um deslocamento pelo grid de células, após um certo número de fases. A estrutura de spaceship mais conhecida é o Glider, que se desloca após o quarto período de vida. A representação de variação e deslocamento do glider pode ser vista na Figura 3.
- **Guns:** são estruturas que produzem fluxos de spaceships, por exemplo de gliders. A Figura 4 mostra um Glider Gun, os gliders podem ser vistos se deslocando na diagonal baixa à esquerda da imagem.
- **Methuselahs:** são estruturas que crescem durante muitas gerações, até estabilizarem. Um exemplo de methuselah é o R-pentomino, que estabiliza após 1103 gerações.
- **Puffers e Rakes:** são estruturas que se deslocam, deixando rastros de outras estruturas para trás(puffers) ou emitindo gliders(rakes). Um exemplo de Puffer pode ser visto na Figura 5, a estrutura se move verticalmente, no lado direito da imagem está a base e no lado esquerdo está o rastro produzido.

4 Objetivos

O objetivo do projeto foi implementar em código C++, quatro versões do Jogo da Vida de Conway, são elas:

- **Versão matriz (serial e paralela)**: o grid de células é um array bidimensional de tamanho fixo. Cada posição guarda um valor boleano, indicando se a célula está viva ou morta (0 ou 1), um laço percorre todas as posições em cada geração.

- **Versão tabela de hash (serial e paralela)**: o grid de células é representado por uma estrutura baseada em tabela de hash, contendo apenas as coordenadas das células vivas. A cada geração, o algoritmo atualiza uma outra estrutura, também baseada em tabela de hash, para contar vizinhos e gerar novos conjuntos, evitando percorrer células mortas e reduzindo o custo quando o padrão fica pouco denso.

Foram geradas métricas de desempenho para comparação de benchmark entre as diferentes versões, analisando variações de entrada como número de gerações e dimensão do grid de células.

5 Grid em matriz

5.1 Implementação serial

A versão serial em `serial-matrix.cpp` recebe um arquivo como entrada, que lista número de gerações, número de linhas da matriz, número de colunas da matriz e uma sequência de coordenadas no formato (x,y), marcando as células vivas inicialmente no grid. O programa armazena a geração atual em um vetor, processa sequencialmente cada posição da matriz, verificando os índices vizinhos de cada célula e atualizando o estado conforme as regras do Jogo da Vida de Conway, calcula e armazena o resultado em um vetor que representa a geração seguinte. Foi adicionado um padding ao redor do grid, com duas linhas e duas colunas extras, para que não houvesse problemas relacionados ao processamento das células na borda do tabuleiro.

5.2 Implementação paralela

A versão paralela em `parallel-matrix.cpp` utiliza a diretiva `for static` do OpenMP, que indica ao runtime a tentativa de divisão igualitária de processamento das linhas do grid entre as threads do bloco paralelo.

Cada thread processa uma linha do vetor que guarda a geração atual, verifica os vizinhos e produz a linha equivalente no vetor que armazena a geração seguinte. Como cada thread processa e escreve uma linha diferente dos vetores, não é necessário se preocupar com a ocorrência de condições de corrida.

O processamento se dá em 3 fases (considerando paralelização)

- Mapeamento de cada uma das células vivas para suas 6 células vizinhas.
- Redução, onde juntamos os contadores resultando o contador final de cada uma das células com vizinhos vivos.
- Calculo do próximo estado

Os principais desafios para a paralelização se chamam unordered_set e unordered_map, por causa que ambas as estruturas de dados não são thread-safe, então acontece segmentation fault ao executar inserções simultâneas. Isso causou diversos gargalos de memorias sendo eles:

- **Iteração:** Por guardarmos as informação dos vivos num unordered_map, que não é possível acesso aleatório, logo não é não é possível atribuir uma faixa de index para cada thread e

seria necessário acessar via iterador. A solução para resolver esse problema, foi criar um vetor auxiliar de todos os vivos, onde é possível acesso aleatório, portanto, é usado o vetor para cada uma das threads iterarem, e o set para verificação se célula está viva em O(1) amortizado.

- **Criação estado inicial:** Quando executamos com um grid grande, esse é um dos principais motivos de a solução com matriz ser melhor que a com set, por causa que pelo fato de que inserir na tabela de hash um valor novo é demorado. Solução encontrada para isso foi implementar uma redução em arvore. Onde, primeiros registramos todas as células vivas, e então criamos um vetor de sets do mesmo tamanho do numero de threads, e então fazemos uma redução em arvore dos sets. Isso se dá pelo fato de que juntar 2 sets de tamanho n, é mais rápido do que inserir $2n$ valores num set.

- **Descobrir quais células são vizinhas de alguma viva:** Se considerássemos que todas as células estão no map, seria simples contar, bastaria um atomic_int para cada uma das celulas, porém é preciso adicionar no map dinamicamente, então essa parte da adição no map fica custosa. E criar uma região critica na adição não soluciona o problema, porque muitos valores precisam ser inicializados no minimo $1/8$(estado onde todos vizinhos de todas celulas existem) das operações são inicializações. Este foi o maior problema enfrentado. Onde tivemos duas soluçÕes principais.

  - Primeiro, o que fizemos é semelhante a solução da criação do estado inicial, cada thread possui um vetor proprio onde adiciona

5.3 Resultados de benchmark

5.3.1 Entradas com quantidade fixa de gerações e grid aleatório

Primeiramente, foram analisadas entradas que variam a dimensão do grid de células, mas mantém o número de gerações fixo. As coordenadas de células inicialmente vivas são definidas aleatoriamente. A quantidade de threads no bloco paralelo foi configurada para 12 threads.

Fixando a entrada em 50 gerações e variando a dimensão do grid em 1.000, 10.000, 1.000.000 e 100.000.000, foram obtidos os resultados de tempo médio de execução, tempo máximo de execução, tempo mínimo de execução e desvio padrão, vistos nas Figuras 6, 7, 8 e 9. Na Figura 10 vemos o gráfico de speedup para a configuração analisada, podemos observar o crescimento do speedup conforme a dimensão do grid aumenta, esse comportamento pode ser explicado pelo aumento da granularidade do problema, garantindo a compensação do custo de paralelização. Esse padrão se repete no gráfico de eficiência, visto na Figura 11. Na Figura 12, vemos o tempo de execução paralela se mantendo muito próximo ao tempo de execução serial para tamanhos de grid menores, e a compensação da paralelização a partir de tamanhos de grid maiores. Um aspecto que pode ser destacado em relação à análise, é a necessidade de se ter grids de dimensões muito grandes, para que a paralelização comece a apresentar resultados favoráveis de performance.

6 Grid em tabela de hash

6.1 Implementação serial

A versão serial, em `serial-set.cpp`, recebe o mesmo formato de entrada descrito anteriormente. O tabuleiro é representado apenas pelo conjunto das coordenadas das células vivas, armazenado em um `std::unordered_set`. Para cada geração, o programa percorre o conjunto de vivos, gera até oito vizinhos para cada célula e acumula a contagem dessas vizinhanças em um `std::unordered_map` <coordenada,,contagem>. Ao final da varredura o mapa é percorrido, se a contagem é 3, a célula nasce, se é 2 e a célula já estava viva, ela sobrevive. O resultado é copiado para um novo `unordered_set`, evitando percorrer células mortas e tornando o algoritmo proporcional apenas ao número de células vivas em cada geração.

6.2 Implementação paralela

Na versão paralela, `parallel-set.cpp`, o conjunto de vivos é copiado para um `std::vector` para permitir iteração por índices. Um `#pragma omp parallel for schedule(static)` distribui esse vetor entre as threads. Cada thread mantém mapas locais de contagem. Ao término do laço, usa-se uma redução manual para fundir esses mapas em um único contêiner global. O mesmo ocorre com o conjunto das novas células vivas. Como cada thread escreve em estruturas próprias, evita-se contenção e condições de corrida, restando apenas o custo de fusão dos mapas.

6.3 Resultados de benchmark

6.3.1 Entradas com quantidade fixa de gerações e grid aleatório

Repetimos o experimento anterior: 50 gerações, dimensões de 1.000, 10.000, 1.000.000 e 100.000.000 células, usando 12 threads. As Figuras 13, 14, 15 e 16 apresentam as métricas de tempo obtidas. Os gráficos de tempo, speedup e eficiência estão nas Figuras 19, 17 e 18, respectivamente.

Observa-se um comportamento oposto ao da versão em matriz. Para dimensões pequenas o tempo absoluto é muito baixo e o overhead de criar threads e fundir contêineres faz o speedup ficar inferior a 1. À medida que o grid cresce e o número de células vivas por geração aumenta, a granularidade compensa o overhead e o speedup alcança valores maiores. Entretanto, quando o grid chega a 100.000.000 posições, o padrão aleatório se torna escasso após poucas gerações, reduzindo novamente o trabalho por thread e fazendo o speedup decair.

Esses resultados confirmam que o modelo esparso é vantajoso quando o número de células vivas permanece proporcional ao tamanho do tabuleiro. Caso a população morra rapidamente (como no padrão aleatório de baixa densidade), o custo fixo de paralelização domina. Vemos um contraste entre o modelo esparso e o modelo denso, no qual todo o tabuleiro é processado a cada geração e a paralelização só compensa para grids muito grandes.

Apêndice



Figura 1: Exemplos de estruturas still lifes no Jogo da Vida de Conway.

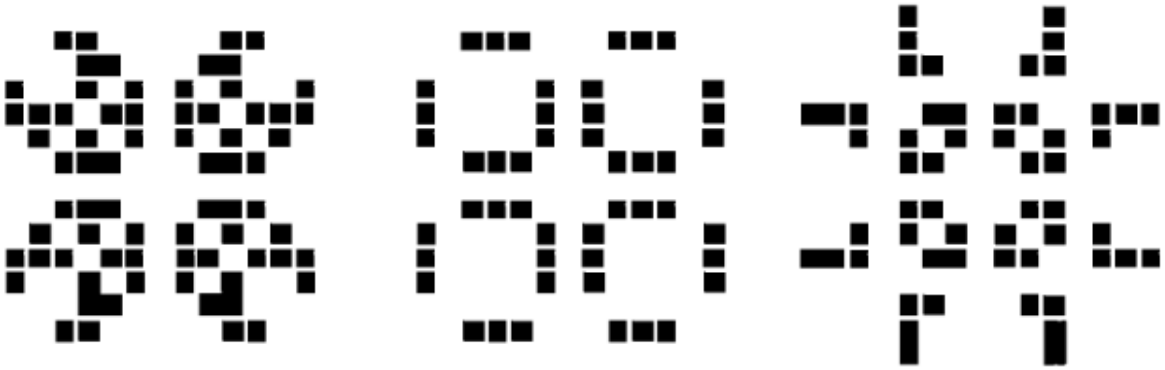


Figura 2: Exemplo de períodos do padrão Pulsar, um oscilator no Jogo da Vida de Conway.



Figura 3: Exemplo de períodos e deslocamento do Glider, um spaceship no Jogo da Vida de Conway.

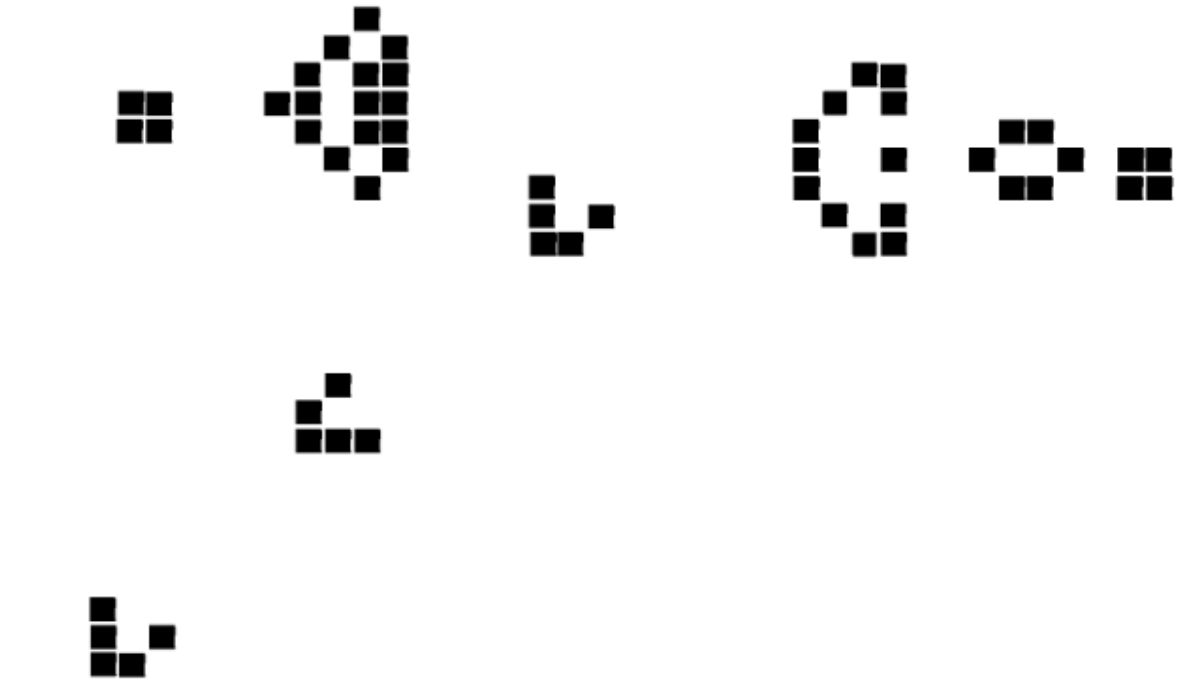


Figura 4: Exemplo de Glider Gun, um gun no Jogo da Vida de Conway.

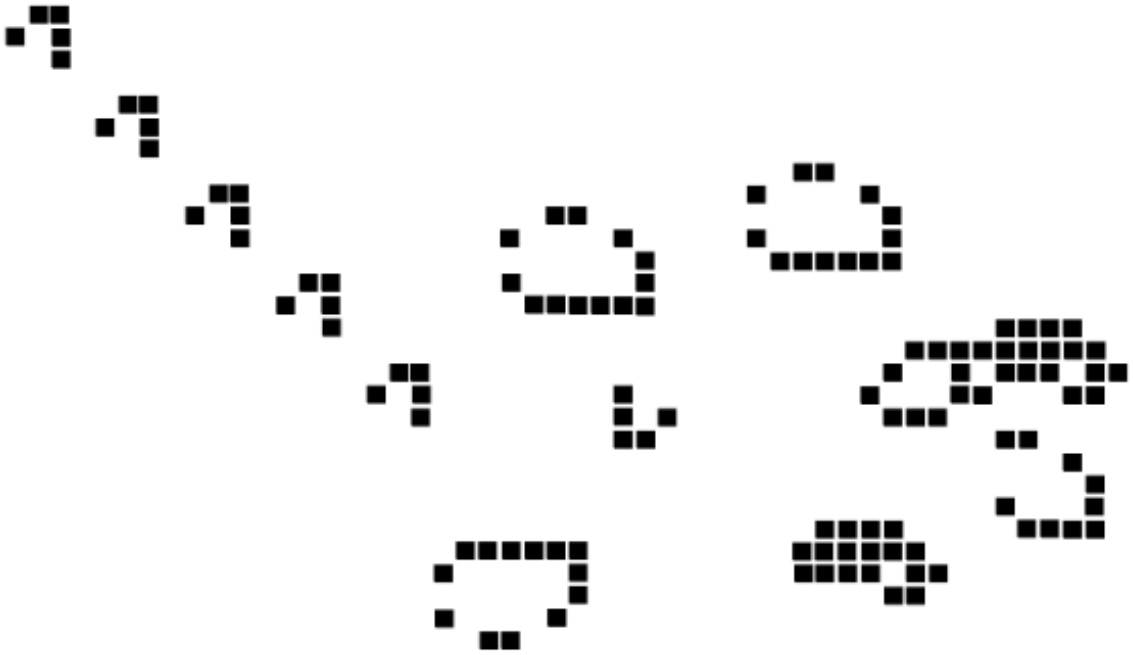


Figura 5: Exemplo de Puffer no Jogo da Vida de Conway.

```
matrix-fix-random:
Dimensão do grid: 1000
Tempo médio de execução serial: (0.0000079 ± 0.0000002) s
Tempo máximo de execução serial: 1.5e-05 s
Tempo mínimo de execução serial: 7e-06 s
Desvio padrão de execução serial: 1.3059862173851609e-06 s
------------------------------------------------------------
Tempo médio de execução paralela: (0.0069 ± 0.0003) s
Tempo máximo de execução paralela: 0.008999 s
Tempo mínimo de execução paralela: 0.000361 s
Desvio padrão de execução paralela: 0.0019823917297042983 s
```

Figura 6: Métricas de tempo obtidas para o grid em matriz com 1.000 células e 50 gerações.

```
matrix-fix-random:
Dimensão do grid: 10000
Tempo médio de execução serial: (0.0000303 ± 0.0000005) s
Tempo máximo de execução serial: 4.6e-05 s
Tempo mínimo de execução serial: 2.7e-05 s
Desvio padrão de execução serial: 3.3335266610603252e-06 s
------------------------------------------------------------
Tempo médio de execução paralela: (0.0069 ± 0.0002) s
Tempo máximo de execução paralela: 0.009007 s
Tempo mínimo de execução paralela: 0.001389 s
Desvio padrão de execução paralela: 0.0014757230818822345 s
```

Figura 7: Métricas de tempo obtidas para o grid em matriz com 10.000 células e 50 gerações.

```
matrix-fix-random:
Dimensão do grid: 1000000
Tempo médio de execução serial: (0.00360 ± 0.00008) s
Tempo máximo de execução serial: 0.006036 s
Tempo mínimo de execução serial: 0.002922 s
Desvio padrão de execução serial: 0.0005833401494839867 s
------------------------------------------------------------
Tempo médio de execução paralela: (0.0071 ± 0.0001) s
Tempo máximo de execução paralela: 0.009205 s
Tempo mínimo de execução paralela: 0.004777 s
Desvio padrão de execução paralela: 0.0010242376278969641 s
```

Figura 8: Métricas de tempo obtidas para o grid em matriz com 100.000 células e 50 gerações.

```
matrix-fix-random:
Dimensão do grid: 100000000
Tempo médio de execução serial: (0.1522 ± 0.0004) s
Tempo máximo de execução serial: 0.168478 s
Tempo mínimo de execução serial: 0.151302 s
Desvio padrão de execução serial: 0.0025361535512661 68 s
------------------------------------------------------------
Tempo médio de execução paralela: (0.0381 ± 0.0003) s
Tempo máximo de execução paralela: 0.045359 s
Tempo mínimo de execução paralela: 0.03363 s
Desvio padrão de execução paralela: 0.0019989145155308668 s
```

Figura 9: Métricas de tempo obtidas para o grid em matriz com 100.000.000 células e 50 gerações.

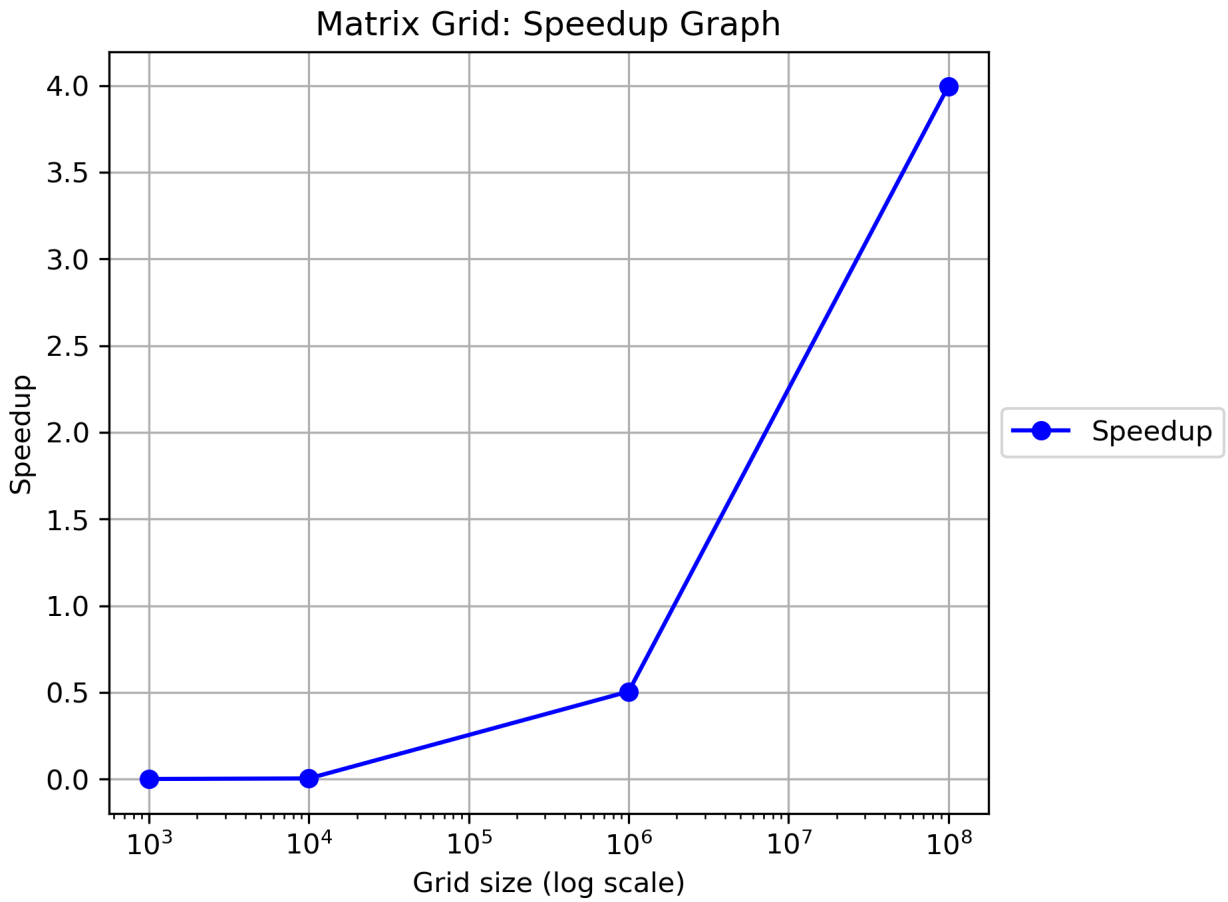


Figura 10: Gráfico de speedup, para o grid em matriz e 50 gerações.

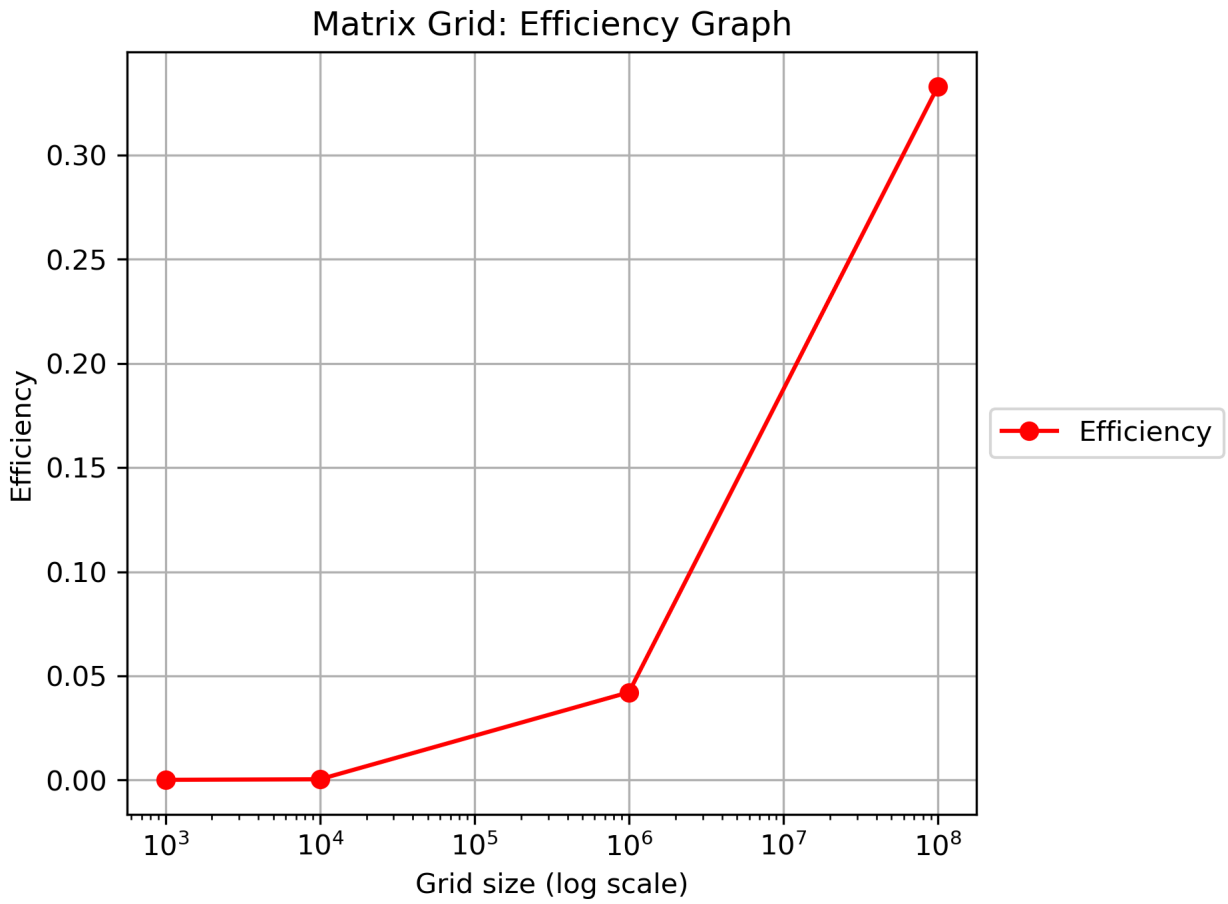


Figura 11: Gráfico de eficiência, para o grid em matriz e 50 gerações.

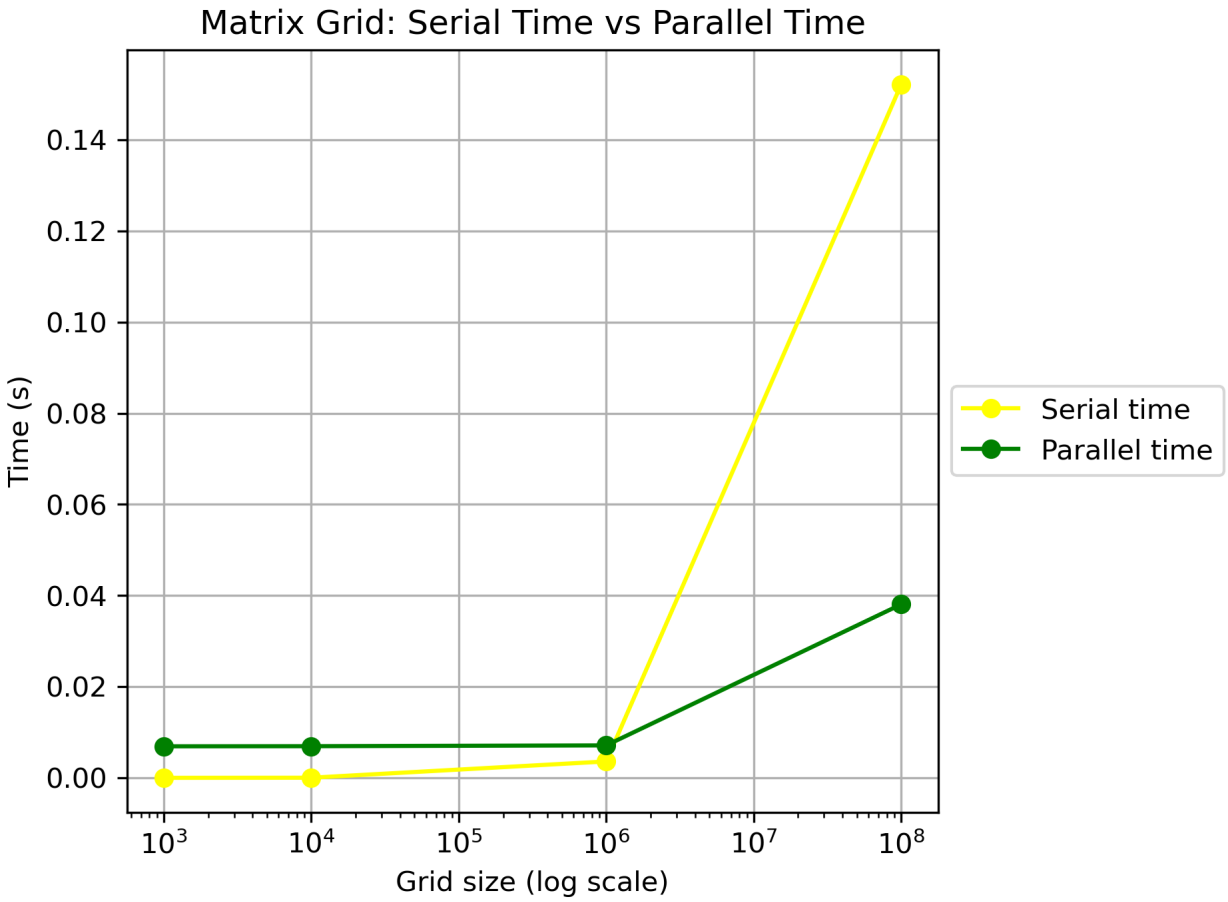


Figura 12: Gráfico de tempo de execução serial e paralela, para o grid em matriz e 50 gerações.

```
set-fix-random:
Dimensão do grid: 1000
Tempo médio de execução serial: (0.0000197 ± 0.0000005) s
Tempo máximo de execução serial: 3.5e-05 s
Tempo mínimo de execução serial: 1.6e-05 s
Desvio padrão de execução serial: 3.3408980828513758e-06 s
------------------------------------------------------------------------
Tempo médio de execução paralela: (0.0000275 ± 0.0000007) s
Tempo máximo de execução paralela: 5.1e-05 s
Tempo mínimo de execução paralela: 2.5e-05 s
Desvio padrão de execução paralela: 4.622596672866885e-06 s
```

Figura 13: Métricas de tempo obtidas para o grid em tabela de hash com 1.000 células e 50 gerações.

```
set-fix-random:
Dimensão do grid: 10000
Tempo médio de execução serial: (0.00050 ± 0.00003) s
Tempo máximo de execução serial: 0.001764 s
Tempo mínimo de execução serial: 0.000316 s
Desvio padrão de execução serial: 0.00019393581309288903 s
------------------------------------------------------------
Tempo médio de execução paralela: (0.00062 ± 0.00003) s
Tempo máximo de execução paralela: 0.001774 s
Tempo mínimo de execução paralela: 0.000449 s
Desvio padrão de execução paralela: 0.00017895707194743 66 s
```

Figura 14: Métricas de tempo obtidas para o grid em tabela de hash com 10.000 células e 50 gerações.

```
set-fix-random:
Dimensão do grid: 1000000
Tempo médio de execução serial: (1.3 ± 0.1) s
Tempo máximo de execução serial: 7.054559 s
Tempo mínimo de execução serial: 0.683058 s
Desvio padrão de execução serial: 0.9887019023252508 s
------------------------------------------------------------
Tempo médio de execução paralela: (0.73 ± 0.04) s
Tempo máximo de execução paralela: 1.401129 s
Tempo mínimo de execução paralela: 0.425106 s
Desvio padrão de execução paralela: 0.287911059414344 14 s
```

Figura 15: Métricas de tempo obtidas para o grid em tabela de hash com 1.000.000 células e 50 gerações.

```
set-fix-random:
Dimensão do grid: 100000000
Tempo médio de execução serial: (0.2 ± 0.2) s
Tempo máximo de execução serial: 11.438382 s
Tempo mínimo de execução serial: 0.001295 s
Desvio padrão de execução serial: 1.6009746712684967 s
------------------------------------------------------------
Tempo médio de execução paralela: (0.2 ± 0.2) s
Tempo máximo de execução paralela: 9.591169 s
Tempo mínimo de execução paralela: 0.001956 s
Desvio padrão de execução paralela: 1.3423873568254134 s
```

Figura 16: Métricas de tempo obtidas para o grid em tabela de hash com 100.000.000 células e 50 gerações.

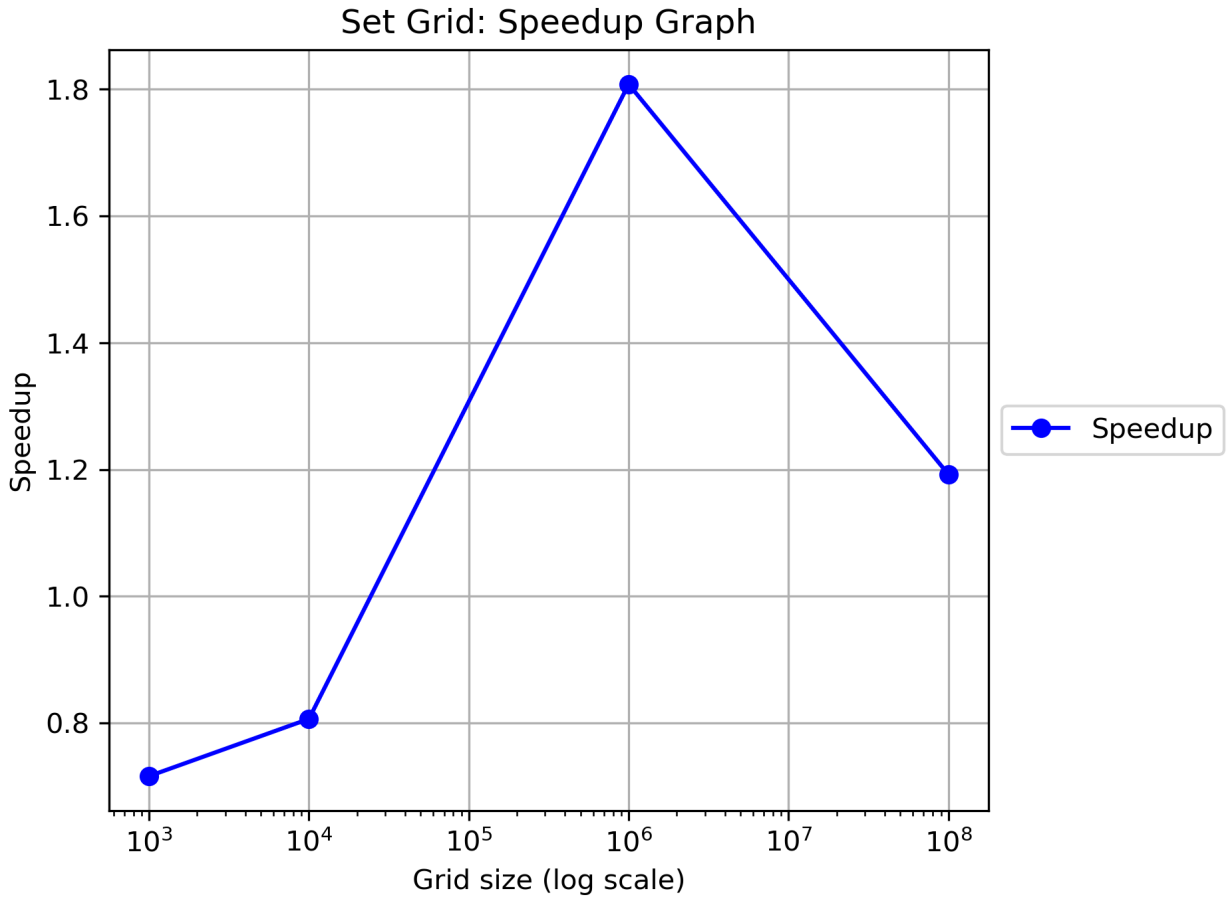


Figura 17: Gráfico de speedup, para o grid em tabela de hash e 50 gerações.

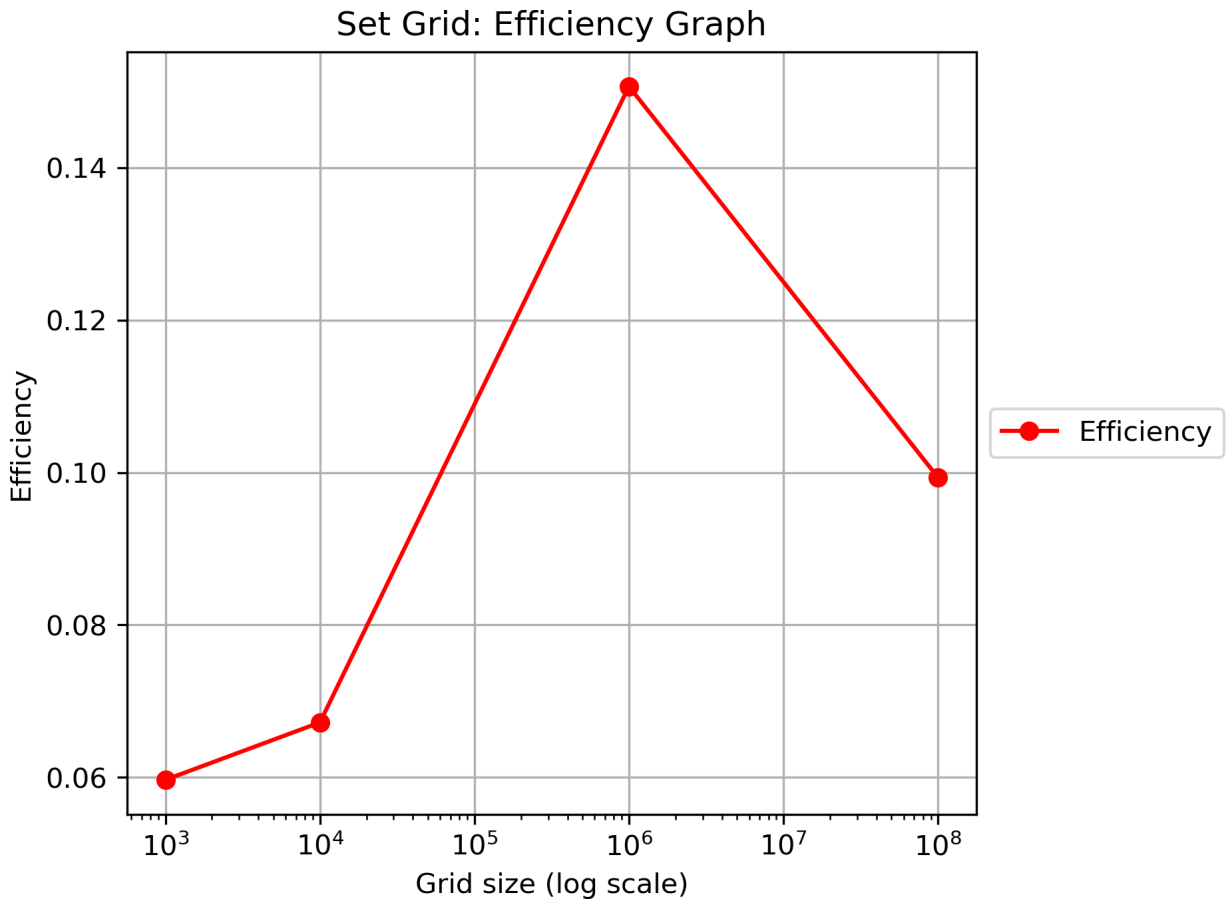


Figura 18: Gráfico de eficiência, para o grid em tabela de hash e 50 gerações.

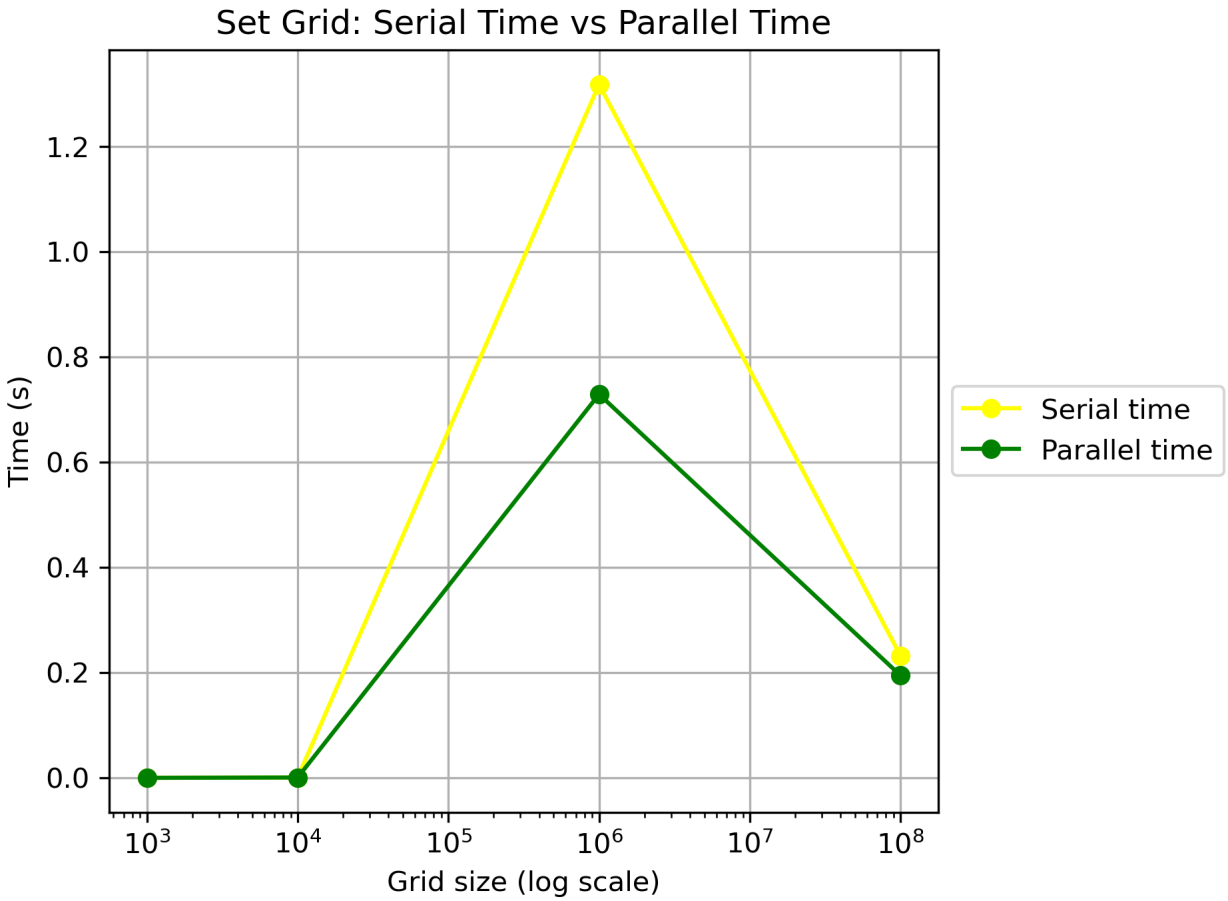


Figura 19: Gráfico de tempo de execução serial e paralela, para o grid em tabela de hash e 50 gerações.

Referências

[1] TIM.LEWIS. About Us. Disponível em: <https://www.openmp.org/about/about-us/>.

[2] SCHIFF, J. L. Cellular Automata. [s.l.] John Wiley & Sons, 2011.